ROM Audit Tool

User Manual

Version 1.4.4c

Batocera • RetroPie • Recalbox

Jason — muckypaws.com

Table of Contents

1 What Is ROM Audit Tool?

ROM Audit Tool tests every game in your collection by actually launching it through its emulator, watching what happens, and recording whether it worked. It does this automatically, with no sitting and watching thousands of games load.

1.1 What it does

- Launches every ROM in your collection one by one through the native emulator
- Detects whether each game loaded successfully from logs and exit codes
- Classifies results: OK, IMPERFECT, ERROR, GENUINE ERROR, MISSING BIOS, MISSING CORE, TIMEOUT
- Captures optional screenshots at launch with annotation and flat-directory output
- Records MD5 or SHA1 checksums to detect silent ROM replacements between runs
- Attempts automatic core fixes for failing ROMs and writes the fix to your config
- Archives error logs per ROM for offline diagnosis
- Discovers and tests ports (Quake, Doom, Cave Story, etc.) on Recalbox and Batocera
- Compares audit runs to surface regressions, improvements and ROM replacements

1.2 What it does not do

- Play games or test gameplay beyond the initial load
- Fix games that have no working emulator on your platform
- Source, download or modify your ROM files

1.3 Supported platforms

Platform	Versions	Notes
Batocera	v36 – v43+	Text adventure games Games using interactive terminal launchers (CON: prefix, e.g. Zork via frotz) cannot be display-tested, as they require interactive stdin unavailable over SSH. These are automatically detected and skipped with a MISSING CORE status and explanatory note. The result is accurate: the ROM itself is fine, the launch method is incompatible with automated testing. Verify the game plays manually from EmulationStation. Full support including autofix, ports and IMPERFECT detection
RetroPie	4.x+ (Bullseye+)	Full support. Stretch/Buster require OS upgrade first
Recalbox	9.x – 10.x	Full support including ports via gamelist.xml discovery

2 Before You Begin

2.1 What you will need

- A device running Batocera, RetroPie, or Recalbox
- SSH access from a computer on the same network
- Python 3.9 or later (check with: python3 --version)

2.2 Terminology

Term	Meaning
Autofix	The tool's automatic attempt to find a working core for a failing ROM
BIOS	System firmware some emulators require, separate from game ROMs
Core	An emulator, the software that runs a ROM
ERROR	The ROM failed to load. The launcher exited with an error, or error text was detected in the logs.
FIXED	The ROM was failing but autofix found a working core and wrote it to your platform config. It will now launch correctly from EmulationStation.
GENUINE ERROR	The ROM failed with every available core combination. Autofix has exhausted all options. Typical causes: wrong ROM set version, corrupt dump, missing parent ROM.
IMPERFECT	Game loads but MAME flags known accuracy issues (colour, sound etc.)
MISSING BIOS	A required BIOS file was not found in the BIOS directory. Install the correct file and recheck.
MISSING CORE	The emulator required to run this ROM is not installed on the device. Install the core via your platform frontend, then recheck.
NEEDS REVIEW	Result cannot be fully trusted without a visual check, typically because FBNeo can silently show a blank error screen with no log evidence of failure.
NO COMBINATIONS	Autofix has no core combinations configured for this system on this platform, so nothing was tried. The system is not yet covered by autofix.
OK	The ROM launched successfully. It started, displayed on screen for the full display window, and exited cleanly with no errors in the logs.
Port	A standalone game running natively on the device, e.g. Quake or Doom
ROM	A game file, a digital copy of a cartridge, disc or arcade board
SSH	Secure Shell, a way to control the device from your computer
System	A game platform, e.g. snes, arcade, n64, ports
TIMEOUT	The emulator took too long to produce a display window and was killed. The game may work normally from the menu. Try --screenshot to check.

3 Installation

3.1 Batocera

Connect over SSH. Default credentials: username root, password linux.

```
ssh root@batocera.local
mkdir -p /userdata/system/rom_audit/modules/{common,platforms}
mkdir -p /userdata/system/rom_audit/scripts
# From your computer:
scp -r rom_audit/ root@batocera.local:/userdata/system/
```

3.1.1 External Storage (USB Drive / Extra Paths)

Batocera can use additional ROM locations beyond /userdata/roms/ via the emulationstation.extrapath setting in batocera.conf. This is commonly used to mount a USB drive containing a larger ROM library.

To configure an extra ROM path in Batocera:

```
# In /userdata/system/batocera.conf
emulationstation.extrapath=/media/usb0/roms
```

ⓘ The audit tool reads emulationstation.extrapath from batocera.conf automatically. Any configured paths that exist on disk are included in the audit alongside /userdata/roms/. No manual configuration of the tool is required.

3.2 RetroPie

Connect over SSH. Default credentials: username pi, password raspberry.

```
ssh pi@retropie.local
mkdir -p ~/RetroPie/rom_audit/modules/{common,platforms}
```

ⓘ On RetroPie, the tool automatically stops EmulationStation before testing and restarts it afterward. Both the standard and -dev branch are detected. If a game is running when the audit starts, it is killed first. No manual intervention is needed.

```
mkdir -p ~/RetroPie/rom_audit/scripts
scp -r rom_audit/ pi@retropie.local:~/RetroPie/
```

3.2.1 External Storage (USB Drive)

RetroPie supports two methods for using USB drives to extend your ROM library:

Method 1 — USB ROM Service (copy)
RetroPie's built-in usbromservice automatically copies ROMs from a USB drive into /home/pi/RetroPie/roms/ when the drive is inserted. These ROMs appear in the standard location and are discovered automatically with no additional configuration.

Method 2 — USB Mount (link, for large libraries)
For libraries too large to fit on the SD card, RetroPie's exfat-automount service mounts the USB drive and creates a /home/pi/RetroPie-mount symlink pointing to it. The audit tool automatically detects this location and includes ROMs from both the SD card and the USB drive in the same audit run.

```
# Verify the mount is active
ls /home/pi/RetroPie-mount/roms/
```

ⓘ If /home/pi/RetroPie-mount/roms/ exists when the audit starts, those ROMs are included automatically. No configuration needed. The tool deduplicates across both locations so a ROM present on both SD card and USB is only tested once.

3.3 Recalbox

Connect over SSH. Default credentials: username root, password recalboxroot.

```
ssh root@recalbox.local
mkdir -p /recalbox/share/system/rom_audit/modules/{common,platforms}
mkdir -p /recalbox/share/system/rom_audit/scripts
scp -r rom_audit/ root@recalbox.local:/recalbox/share/system/
```

3.3.1 Dual ROM Storage

Recalbox stores ROMs in two locations that are both scanned automatically:

- /recalbox/share/roms/, your writable user area. ROMs you add go here.
- /recalbox/share_init/roms/, read-only factory ROMs included with Recalbox (free games, demos, homebrew).

ⓘ Both locations are scanned and merged into a single audit run automatically. ROMs present in both locations are only tested once, the user copy takes priority. No configuration is required.
ⓘ On Recalbox, always use /recalbox/share/system/ as files stored elsewhere on the rootfs are wiped on system updates.

3.4 Check prerequisites

Run this on installing/copying the tool to your target system

```
cd /userdata/system/rom_audit           # Batocera
python3 scripts/prereqs.py
```

```
cd /recalbox/share/system/rom_audit     # RecalBox
python3 scripts/prereqs.py
```

```
cd /home/pi/RetroPie/rom_audit          # RetroPie
python3 scripts/prereqs.py
```

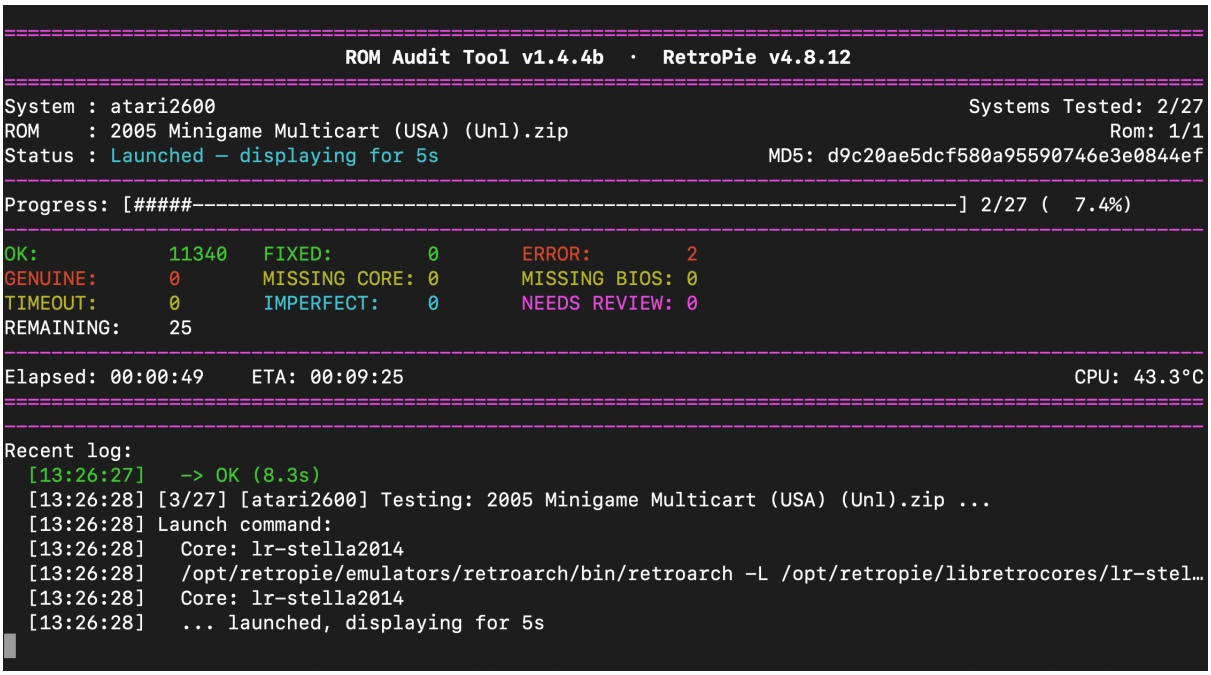
```
Python 3.11.8 on Linux aarch64

Python
──────
[PASS]  Python version                          3.11.8

Python standard library
───────────────────────
[PASS]  All required stdlib modules             present

Platform detection
──────────────────
[PASS]  Platform                                Recalbox 10.0.8

System tools
────────────
[PASS]  pkill                                   available
[PASS]  python3 on PATH                         /usr/bin/python3

ROM Audit Tool structure
────────────────────────
[PASS]  rom_audit.py                            /recalbox/share/system/rom_audit/rom_audit.py
[PASS]  All modules present                     9 module(s) found

Recalbox specific
─────────────────
[PASS]  recalbox.conf                           /recalbox/share/system/recalbox.conf
[PASS]  ROMs directory                          /recalbox/share/roms (109 system(s))
[PASS]  Built-in ROMs                           /recalbox/share_init/roms (108 system(s))
[PASS]  Libretro cores                          130 core(s) in /usr/lib/libretro
[PASS]  emulatorlauncher                        /usr/lib/python3.11/site-packages/configgen/emulatorlauncher.py
[PASS]  Log directory                           /recalbox/share/system/logs
[PASS]  BIOS directory                          /recalbox/share/bios (616 file(s))
[PASS]  Screenshot tool                         fbgrab available
[PASS]  Framebuffer                             /dev/fb0 present
[PASS]  Disk space                              22330 MB free at /recalbox/share/system

──────────────────────────────────────────────────
Results: 17 passed, 0 warnings, 0 failed (17 checks)

All checks passed — ready to audit.
```

4 Your First Audit

4.1 Start small

Test on a single system with a small limit first to confirm everything is working:

```
python3 rom_audit.py --system atari2600 --limit 5
```



4.2 Full audit

```
python3 rom_audit.py
```

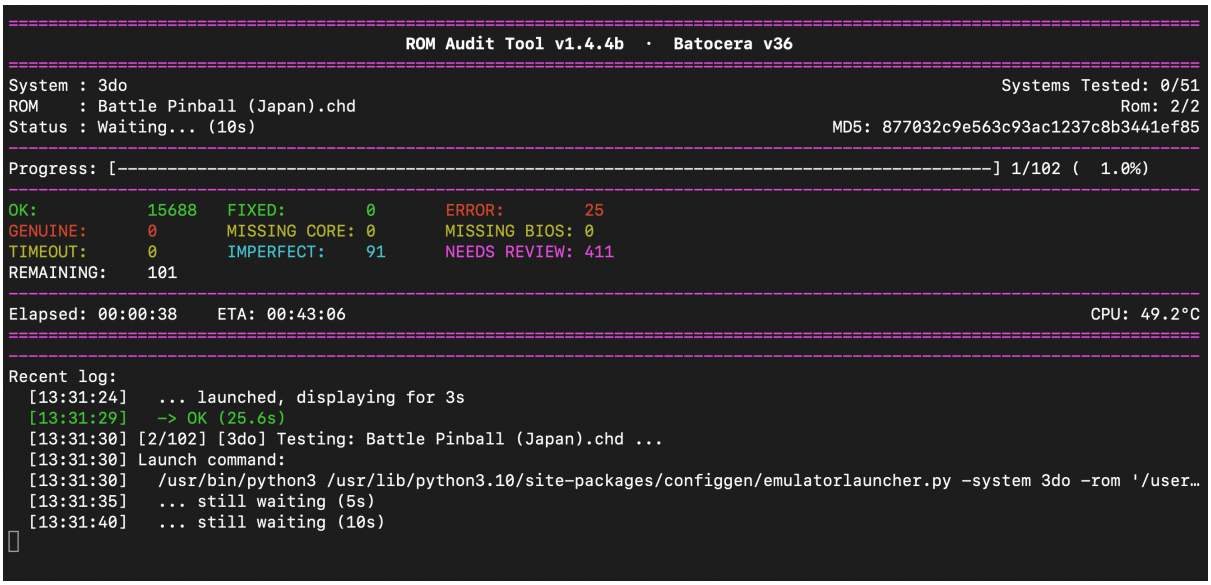
⚠ **The tool stops EmulationStation before testing on RetroPie and Recalbox. You will not be able to use the gaming interface during the audit. See Section 16 for running safely overnight.**

4.3 Key first-run options

Flag	Example	What it does
--system	--system mame	Audit only one system
--limit	--limit 20	Test only N ROMs per system
--no-dashboard	(none)	Plain log output, better for SSH
--test	--test pacman.7z	Test a single named ROM
--exclude	--exclude ports	Skip a system entirely

5 Reading the Dashboard

The live dashboard redraws every 0.5 seconds. All counters update in real time.



Item	Meaning
System / ROM	Currently being tested
Status	What the tool is doing right now (live)
Systems Tested	How many systems have been tested in this run
ROM x/y	Progress of ROMS being tested for this system
MD5/SHA1	If available shows the Checksum for this rom.
Progress Bar	Progress of the testing this run.
OK	ROMs that loaded successfully
FIXED	ROMs fixed automatically by autofix
ERROR	ROMs that failed to load
GENUINE	ROMs that failed every available core combination
MISSING CORE / BIOS	Required emulator or firmware not installed
TIMEOUT	Emulator took too long to respond
IMPERFECT	ROMs that load but have known MAME accuracy warnings
NEEDS Review	ROMs that need user review, either via the CSV, Logs or Screenshots
CPU	CPU Temperature
ETA	Estimated time remaining based on recent test speeds
LOG	Recent Log Messages on a rolling basis.

ⓘ Use --no-dashboard for plain timestamped log output. Recommended when running over SSH in a tmux session.

6 Understanding Results

6.1 Status reference

Status	Meaning	Recommended action
OK	Game loaded successfully	None needed
FIXED	Autofix found a working core	None needed, fix is permanent
IMPERFECT	Game loads but MAME reports known accuracy issues (colour, sound, timing). Playable but not arcade-perfect.	Keep or quarantine with --include-imperfect. Review screenshots.
NEEDS REVIEW	Result came from a core that may mask failures (FBNeo), or ROM has unverified dump quality (NO GOOD DUMP KNOWN / ROM NEEDS REDUMP). Not confirmed broken, but not confirmed good without a screenshot heuristic check.	Check the screenshot, or rerun with --heuristic for an automatic pixel-based check.
ERROR	Game failed to load	Try --autofix, then check error logs
GENUINE ERROR	Failed with every available core	Wrong ROM version, corrupt file, or dump issue
MISSING BIOS	Required BIOS file not found	Install the correct BIOS file in the BIOS directory
MISSING CORE	Required emulator not installed	Install the core via your frontend
TIMEOUT	Emulator took too long to respond	May work normally from the menu, try --screenshot to check

6.2 Viewing the summary

```
python3 scripts/summary.py
python3 scripts/summary.py --sort errors    # Worst systems first
```

```
# python3 scripts/summary.py
Reading: rom_audit.csv

System           Total  OK+Fixed Active Errors  MISSING CORE
------------------------------------------------------------
240ptestsuite       11        11            0            -
amiga600             1         1            0            -
amstradcpc           4         4            0            -
apple2               3         3            0            -
apple2gs             2         2            0            -
atari2600            4         4            0            -
atari5200            1         1            0            -
atari7800            3         3            0            -
atari800             3         3            0            -
c64                 10        10            0            -
colecovision         1         1            0            -
gamegear             1         1            0            -
gb                   4         4            0            -
gba                  9         9            0            -
gbc                  3         3            0            -
intellivision        3         3            0            -
lynx                 2         2            0            -
mame                 4         4            0            -
megadrive           11        11            0            -
msx1                 5         5            0            -
msx2                 2         2            0            -
nes                 16        16            0            -
oricatmos            2         2            0            -
pcengine             2         2            0            -
ports               12        12            0            -
sg1000               1         1            0            -
snes                 1         1            0            -
solarus              1         0            1            1
thomson              1         1            0            -
tic80              189       189            0            -
------------------------------------------------------------
TOTAL              312       311            1            1

  Overall: 311/312 OK (99.7%)  -  1 outstanding
```

6.3 Listing errors

```
python3 scripts/romlist.py --errors
python3 scripts/romlist.py --system mame
python3 scripts/romlist.py --imperfect      # List IMPERFECT ROMs
python3 scripts/romlist.py --needs-review   # List NEEDS REVIEW ROMs
```

```
# python3 scripts/romlist.py --errors
# errors

[uzebox]  (3 ROM(s))
----------------------
"Candle Burner.uze", [md5:497903b2bedbe9628e2d9cb752c97b6c], "Process exitcode: 1"
"Columns.uze", [md5:99666b7c93aca68c5522c2e7b412f98b], "Process exitcode: 1"
"UzeBeats.uze", [md5:d4e9879d3366194f485bfac4583bb397], "Process exitcode: 1"

[wasm4]  (3 ROM(s))
---------------------
"bombfighters.wasm", [md5:3db1c45ee780b31975e5866bda3072d3], "Process exitcode: 1"
"seed-creator-showcase.wasm", [md5:513f1ab8fe332a2908d8952f244a7fb0], "Process exitcode: 1"
"spunky.wasm", [md5:7f5cbd2ef69d2f8ebc72e631e239325e], "Process exitcode: 1"

  6 ROM(s) listed.
```

7 IMPERFECT ROMs

IMPERFECT is a special status for ROMs that load and run but where MAME's internal driver database flags known emulation shortcomings. This is common in arcade ROM sets where the original hardware used custom chips that are difficult to emulate precisely.

7.1 What triggers IMPERFECT

- Colour palette inaccuracies, the game runs but colours may be slightly wrong
- Sound emulation gaps, some sound channels or effects may be missing or incorrect
- Graphics emulation issues such as sprite priority, transparency or scrolling defects
- Timing issues, where game speed may differ slightly from the original hardware

These are MAME's own honesty flags, displayed as a warning screen at launch. They do not indicate a bad dump or a missing file.

7.2 IMPERFECT vs ERROR

	IMPERFECT	ERROR
Game loads	Yes	No
Game is playable	Yes (with caveats)	No
Cause	Known emulation inaccuracies	Failed to initialise or crashed
Action	User discretion	Fix, replace, or quarantine
Included in --cleanup	Only with --include-imperfect	Yes, by default

7.3 Quarantining IMPERFECT ROMs

By default, --cleanup ignores IMPERFECT ROMs because they are playable. If you want a pixel-perfect arcade-accurate collection and prefer to remove them:

```
# Preview what would be moved
python3 rom_audit.py --cleanup --action move \
    --include-imperfect --dry-run

# Move to quarantine
python3 rom_audit.py --cleanup --action move --include-imperfect

# Or just list them for manual review
python3 scripts/romlist.py --imperfect
```

ⓘ IMPERFECT ROMs are still counted in the totals and shown on the dashboard. They will not be included in autofix attempts since the issue is in the emulation core itself, not the ROM file.

8 Screenshot Capture

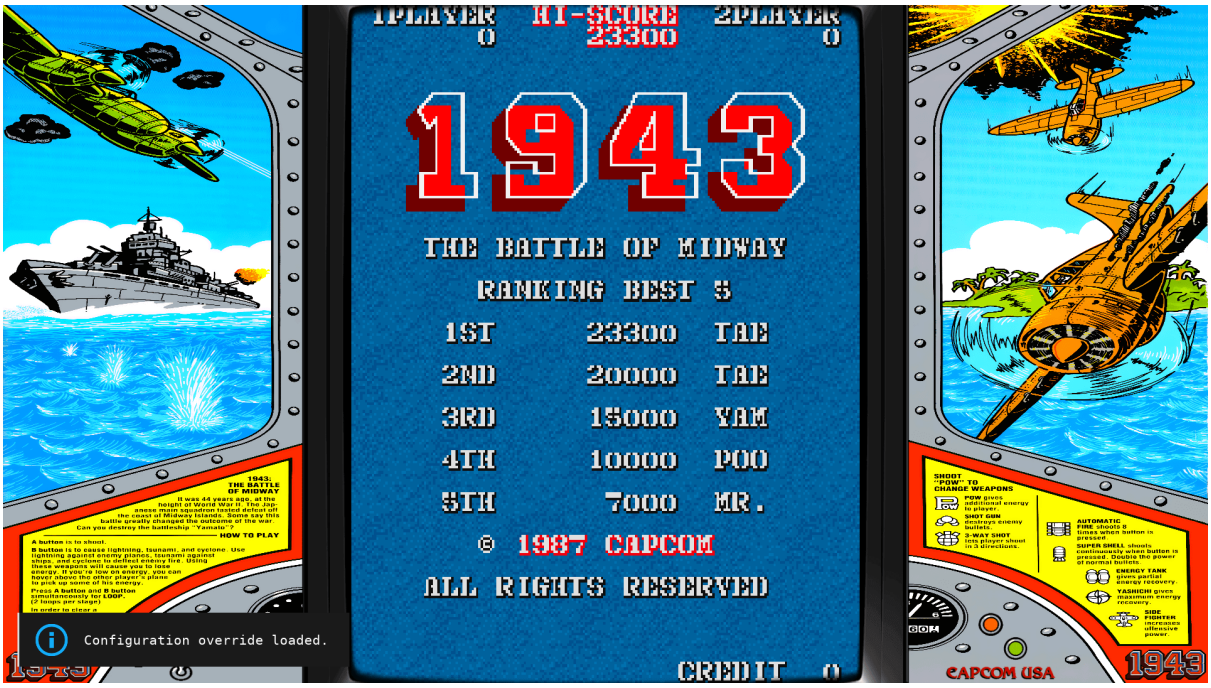
Screenshots capture what is on screen a few seconds after a ROM launches. This is useful for confirming a game actually rendered correctly, or catching games that silently show an error screen rather than the title.

8.1 Basic screenshot

```
python3 rom_audit.py --system atari2600 --screenshot
python3 rom_audit.py --test pacman.7z --system mame --screenshot
```

Screenshots are saved alongside error logs:

```
audit_logs/atari2600/ADVNTURE.BIN/atari2600_ADVNTURE.BIN_screenshot.png
```



8.2 Annotating screenshots

Add --annotate to burn the system name, ROM filename and timestamp directly into the image:

```
python3 rom_audit.py --system mame --screenshot --annotate
```

> ⓘ *Annotation uses libass on Recalbox and ffmpeg drawtext on Batocera. Font size scales automatically to the image resolution.*

8.3 Flat screenshot directory

Save all screenshots in one folder, named per ROM, for easy bulk export:

```
python3 rom_audit.py --screenshot --screenshot-flat
# Saves as: audit_logs/screenshots/mame_pacman.7z.png
```

8.4 Screenshot delay

Add extra wait time before capture for systems with long BIOS splash screens:

```
python3 rom_audit.py --system neogeo --screenshot --screenshot-delay 8
```

System	Suggested delay	Reason
neogeo	15 seconds	SNK BIOS splash
naomi / atomiswave	15 seconds	Sega BIOS initialisation
psp	20 seconds	UMD image loading
n64	15 seconds	mupen64plus startup
mame	10 seconds	MAME attract mode / warning screens

ⓘ *Screenshots are the best way to catch IMPERFECT ROMs where the game runs but shows a MAME warning screen or incorrect visuals.*

9 Checksums

Record a hash per ROM to detect silent file replacements between runs:

```
python3 rom_audit.py --checksum md5
python3 rom_audit.py --checksum sha1
```

9.1 Checksums are sticky, and validated automatically once present

You only need `--checksum` the first time. Once a ROM has a checksum on record, every later test of it, whether recheck, autofix, a plain `--new` pass, or anything else, keeps that checksum rather than wiping it back to blank just because that particular run did not also pass `--checksum`.

Once a checksum is on record, passing `--checksum` again on a later run validates the file against the stored hash. If the content no longer matches, you will see a clear warning in the log and a CHECKSUM MISMATCH note in the CSV. Validation only runs when `--checksum` is explicitly passed, so plain recheck or autofix runs preserve existing checksums without re-reading the file, avoiding silent multi-minute stalls on large CD images.

The recorded checksum deliberately does NOT update itself when a mismatch is found, the mismatch keeps being reported on every subsequent run until you explicitly pass --checksum again, which is treated as you deliberately accepting the new file as the new baseline. This is intentional: a mismatch that quietly resolved itself after one report could mean a real, ongoing problem goes unnoticed if you do not happen to read that one run's log closely.

ⓘ *If you see CHECKSUM MISMATCH on a ROM you have not deliberately replaced or updated, treat it the same as the SpyHunter case in Troubleshooting and investigate the file itself before assuming the tool got something wrong. A ROM that "broke" with no other explanation and a changed checksum is evidence of something happening at the storage layer, not a detection bug.*

9.2 Populating checksums without testing

If you want every ROM in your CSV checksummed straight away, rather than waiting for each one to come up for a normal test, scripts/compute_checksums.py reads the file on disk and fills in the checksum column directly — no launching, no testing, just hashing:

```
python3 scripts/compute_checksums.py
python3 scripts/compute_checksums.py --system mame --force
```

By default it only fills in ROMs that have no checksum yet, so running it again after adding new ROMs only does new work. `--force` recomputes everything, including ROMs that already have one, worth doing periodically across a whole collection as a deliberate integrity sweep, separate from the automatic per-test validation above.

Compare two runs and flag replacements:

```
python3 scripts/compare.py before.csv after.csv --checksum
```

Scenario	Meaning
Same status, different checksum	File was replaced between runs, intentional or suspicious
Status improved AND checksum changed	Looks fixed because the ROM was replaced, not the core
Checksums match	Confirmed identical file, any status change is genuine

10 Ports

Ports are standalone games that run natively on your device, including Quake, Doom, Prince of Persia, Cave Story and others. The tool discovers and tests these automatically on Recalbox and Batocera.

```
python3 rom_audit.py --system ports
python3 rom_audit.py  # Ports included in full audit automatically
```

10.1 How port discovery works

- Scans each port subdirectory for a gamelist.xml manifest
- Reads the <path> tag to find the actual launch file
- Determines the correct emulator and core automatically from file type or name
- Skips ports requiring user-supplied proprietary files (roms_needed.txt present)
- Skips ports targeting native Windows executables (.exe, .bat, .com)

10.2 Emulator resolution

File type	Port examples	Emulator
.pak	Quake	libretro / tyrquake
.wad	Doom, Sigil	libretro / prboom
.pk3	Wolfenstein 3D	libretro / ecwolf
.game	2048, Gong, Mr. Boom	libretro (core from file content)
.zip / directory	Prince of Persia	sdlpop
Directory	Cave Story	libretro / nxengine

ⓘ *Standalone ports (Prince of Persia, Theme Hospital) may not be visible on screen during testing due to display ownership when EmulationStation is stopped. Their OK/ERROR result is still correct.*

11 Fixing Problems with Autofix

Autofix tries alternative emulator cores for failing ROMs and writes the working configuration so the game always launches correctly from your frontend in future.

```
python3 rom_audit.py --recheck --autofix --system mame
python3 rom_audit.py --recheck --autofix
```

Autofix also runs automatically on MISSING CORE results, not only ERROR, which is useful on Recalbox where a ROM can be rejected before the emulator ever launches because the default core does not support that file's extension. Other installed cores that do support it still get a fair attempt.

While autofix runs you will see each core it tries and the result of that specific attempt, for example:

```
  [1/4] Trying: libretro / mame2003_plus
    Result: ERROR (6.1s) Process exitcode: 1
  [2/4] Trying: libretro / mame2010
    Result: OK (6.2s)
```

This makes it possible to see exactly why earlier cores were rejected before the one that worked, useful context if you want to understand a fix rather than just accept it.

11.1 Where the fix is written

Platform	Config written to
Batocera	mame["pacman.7z"].core=mame078plus in batocera.conf
RetroPie	arcade_pacman = "lr-mame2003" in emulators.cfg
Recalbox	pacman.zip.recalbox.conf sidecar file next to the ROM

11.2 GENUINE ERROR — when autofix gives up

If a ROM fails every available core combination, it is marked GENUINE ERROR/NO COMBINATION. This means autofix has exhausted all options. Possible causes:

- Wrong ROM set version for your cores (e.g. MAME 0.139 ROMs with MAME 0.78 core)
- Corrupt or incomplete dump, with missing files within the archive
- ROM requires a BIOS that is installed but the wrong version
- Parent ROM missing for a clone ROM
- Intentional bad dump flagged in the No-Intro or TOSEC database

ⓘ Error logs for GENUINE ERROR ROMs are archived in audit_logs/{system}/{rom}/ and contain the full emulator output at the time of failure.

11.3 When autofix cannot fully trust its own result

FBNeo has a confirmed, specific weakness: on an unrecognised or bad dump it can silently show an error screen with no error text logged anywhere. It launches cleanly, runs for the full display window, gets killed as expected, and looks identical in the logs to a genuine success. If autofix tries FBNeo as a candidate core and it reports OK, that OK cannot be trusted the same way a normal candidate's OK can, because the very thing that would normally prove success (clean logs, no errors) is exactly what this specific failure mode also produces.

When this happens, the result is marked NEEDS REVIEW, a status of its own distinct from OK/FIXED and from a confirmed ERROR, specifically meaning “not confirmed broken, but not confirmed good either.” A forced screenshot is captured for that specific attempt regardless of whether --screenshot was set for the run, and the screenshot path is included right there in the notes:

```
mame,bandit.zip,NEEDS REVIEW,3.5,2026-06-23 19:29:13,,"Fixed:
mame[""bandit.zip""].core=fbneo / emulator=libretro — UNVERIFIED: fbneo can
report OK while showing nothing usable on screen, with no error text logged.
Check the screenshot before trusting this result
(audit_logs/mame/bandit.zip/mame_bandit.zip_review.png)"
```

A quick way to find every result this has happened to across a whole audit:

```
python3 scripts/romlist.py --needs-review
```

11.4 --heuristic: automated screen analysis

--heuristic has two levels.

Level 1 (--heuristic alone) analyses the forced verification screenshot whenever FBNeo is accepted as a result, whether via autofix or because an existing config entry already points at FBNeo. It checks whether the centre of the frame is overwhelmingly one flat, light colour, the signature of FBNeo’s blank error dialog rather than real game content, and adds the finding to the notes.

COMING SOON

Level 2 (--heuristic 2) goes further: it captures a baseline screenshot once after EmulationStation shuts down and before the first ROM launches, then compares every post-ROM screenshot against that baseline. A high match means the emulator produced no visible output, the screen looks the same as before any ROM ran. If the screen did change, per-system colour rules then check for known failure states:

```
python3 rom_audit.py --recheck --autofix --heuristic
```

...adds "Pixel analysis: 99% uniform light colour in the sampled region — LIKELY BLANK/ERROR SCREEN" (or "likely genuine content" when the frame is not uniform) onto the end of the UNVERIFIED note. It is still a flag, not a verdict — a deliberately minimalist or fading game screen could occasionally trip it the wrong way — but in practice it turns "go look at every screenshot" into "go look at the ones it actually flagged."

ⓘ --heuristic 1 only triggers on the rare FBNeo result and adds negligible time across a full audit. --heuristic 2 triggers on every ROM and adds one screenshot analysis per test, well under a second at 1080p, a few seconds at 4K. For a large collection this is real time; use Level 2 when you want thorough coverage rather than a quick pass. Both levels are probabilistic flags, not verdicts. A deliberately minimalist game screen could occasionally look blank. The tool annotates notes and may downgrade OK to NEEDS REVIEW, but never silently removes a ROM or changes a result without leaving a trace.

12 Managing Your Results

12.1 Retesting ROMs

```
python3 rom_audit.py --recheck                # Retest all failures
python3 rom_audit.py --recheck --system psx   # One system only
python3 rom_audit.py --new                    # Test only untested ROMs
python3 rom_audit.py --recheck-all            # Full fresh baseline including
OKs
python3 rom_audit.py --recheck-all --system naomi  # Fresh baseline one
system
python3 rom_audit.py --test "Silent Hill.chd" --system psx  # Single ROM
```

12.2 Retesting after a date

```
python3 rom_audit.py --recheck --since 2026-05-01
# Retest any ROM last tested before 1 May 2026
```

12.3 Comparing before and after

```
cp rom_audit.csv rom_audit_before.csv
python3 rom_audit.py --recheck
python3 scripts/compare.py rom_audit_before.csv rom_audit.csv
```

12.4 Cleanup and quarantine

```
# Preview only (safe)
python3 rom_audit.py --cleanup --action move --dry-run

# Move ERROR and GENUINE ERROR ROMs to quarantine
python3 rom_audit.py --cleanup --action move

# Also move IMPERFECT ROMs
python3 rom_audit.py --cleanup --action move --include-imperfect

# Delete permanently (irreversible)
python3 rom_audit.py --cleanup --action delete
```

ⓘ Quarantined ROMs go to faultyroms/{system}/ in your ROMs directory. They are removed from EmulationStation on next restart.

13 Helper Scripts Reference

13.1 prereqs.py — System check

```
python3 scripts/prereqs.py
```

13.2 summary.py — Collection health

```
python3 scripts/summary.py
python3 scripts/summary.py --sort errors
python3 scripts/summary.py --system arcade
```

13.3 romlist.py — List ROMs by status

```
python3 scripts/romlist.py --errors
python3 scripts/romlist.py --genuine
python3 scripts/romlist.py --imperfect
python3 scripts/romlist.py --system snes --ok
python3 scripts/romlist.py --errors --out errors.txt
```

13.4 compare.py — Before and after

```
python3 scripts/compare.py old.csv new.csv
python3 scripts/compare.py old.csv new.csv --improved
python3 scripts/compare.py old.csv new.csv --regressed
python3 scripts/compare.py old.csv new.csv --checksum
python3 scripts/compare.py old.csv new.csv --system arcade
```

13.5 filter.py — Search results

```
python3 scripts/filter.py --status ERROR
python3 scripts/filter.py --status IMPERFECT
python3 scripts/filter.py --system mame --status "GENUINE ERROR"
python3 scripts/filter.py --notes "NOT FOUND"
```

13.6 duplicates.py — Find duplicate ROMs

```
python3 scripts/duplicates.py --csv
python3 scripts/duplicates.py --files --paths
```

13.7 compute_checksums.py — Populate checksums without testing

Reads every ROM the CSV already knows about and hashes the file on disk with no launching or testing. Fills in blanks only by default; --force recomputes everything, useful as a periodic whole-collection integrity sweep.

```
python3 scripts/compute_checksums.py --system mame
python3 scripts/compute_checksums.py --force --algorithm sha1
```

13.8 prune_orphaned_entries.py — Clean up stale CSV rows

Finds CSV rows that no longer correspond to an independently-testable ROM, either a file now correctly recognised as a .cue/.uae companion, or a file that has been deleted entirely, and removes them. Reports both categories separately; only acts on --remove.

```
python3 scripts/prune_orphaned_entries.py
python3 scripts/prune_orphaned_entries.py --remove --remove-missing
```

13.9 clear_system_overrides.py — Reset a system's batocera.conf entries (Batocera)

Removes every override for one or more systems from batocera.conf, including the global default and every per-game entry, plus any stale suspended marker. Useful for starting a system completely fresh after a config gets into a state worth not trusting (see the FBNeo per-game entry case in Troubleshooting). Reports what it found before touching anything; always backs up the file before --remove actually changes it.

```
python3 scripts/clear_system_overrides.py --system mame
python3 scripts/clear_system_overrides.py --system mame fbneo --remove
```

13.10 cleanup_orphaned_screenshots.py — Retroactive screenshot cleanup (Recalbox)

A one-off cleanup for screenshots that accumulated in Recalbox's own gallery before screenshot capture was fixed to move rather than copy. Reads the CSV's tested_at timestamps to scope the cleanup to the actual audit window, so it does not touch screenshots from outside that run.

```
python3 scripts/cleanup_orphaned_screenshots.py rom_audit.csv --dry-run
```

14 Complete CLI Reference

14.1 Core flags

Flag	Example	Description
--system	--system mame	Audit only this system (repeatable)
--exclude	--exclude ports	Skip this system
--limit	--limit 10	Test at most N ROMs per system
--test	--test pacman.7z	Test a single named ROM
--no-dashboard		Plain log output, no ANSI dashboard
--no-es-restart		Do not restart EmulationStation after audit

14.2 Retest and filter flags

Flag	Description
--recheck	Retest all non-OK results
--recheck-all	Retest everything including OK and FIXED
--new	Test only ROMs not yet in the CSV
--since YYYY-MM-DD	Recheck ROMs tested before this date
--autofix	Attempt automatic core fixes for failing ROMs

14.3 Screenshot flags

Flag	Description
--screenshot	Capture a screenshot during each ROM test
--annotate	Burn system, ROM name and timestamp into each screenshot
--screenshot-flat	Save to flat directory as system_romname.png
--screenshot-delay N	Wait N extra seconds before capturing
--heuristic	Pixel-based screen analysis. Level 1 (bare --heuristic): analyses the forced verification screenshot when FBNeo is accepted — checks for a blank or error screen and automatically confirms or clears a NEEDS REVIEW result. Level 2 (--heuristic 2): additionally captures a baseline screenshot after EmulationStation shuts down, then compares every post-ROM screenshot against it; also runs per-system colour rules (Atari800 blue power-on, Solarus black, MAME2010 startup window, Apple II @ grid). Level 2 runs on every ROM, not just FBNeo results. An OK that looks blank is downgraded to NEEDS REVIEW; a BLANK with an already non-OK status adds the analysis to the notes. Both levels are probabilistic flags, not verdicts.

14.4 Checksum and cleanup flags

Flag	Description
--checksum md5	Record MD5 hash for each ROM in the CSV
--checksum sha1	Record SHA1 hash for each ROM in the CSV
--cleanup --action move	Move ERROR ROMs to quarantine
--cleanup --action delete	Permanently delete ERROR ROMs
--include-imperfect	Also include IMPERFECT ROMs in --cleanup
--include-needs-review	Also include NEEDS REVIEW ROMs in --cleanup
--dry-run	Preview cleanup without making changes

15 Tips and Tricks

15.1 Run inside tmux for overnight audits

If your SSH connection drops, the audit is killed. tmux keeps it running:

```
tmux new-session -s romaudit
python3 rom_audit.py --no-dashboard
# Detach: Ctrl+B then D
# Reattach later: tmux attach -t romaudit
```

15.2 Resume an interrupted audit

```
python3 rom_audit.py 'Last ROM Name.zip'
python3 rom_audit.py 'Last ROM Name.zip' --system arcade
```

15.3 Audit one system at a time

For large collections, auditing system by system makes it easier to review results and spot patterns:

```
python3 rom_audit.py --system mame
python3 scripts/summary.py --system mame
python3 rom_audit.py --system mame --recheck --autofix
```

15.4 Use screenshots to catch silent failures

Some games launch without error but display a wrong-hardware screen or garbled graphics. Screenshots catch what log analysis cannot:

```
python3 rom_audit.py --system atari800 --screenshot --annotate
# Review audit_logs/atari800/*/*_screenshot.png
```

15.5 Check for ROM set version mismatches before testing

MAME ROMs are version-specific. If you get many GENUINE ERROR results on an arcade system, check which ROM set version your cores expect:

```
# MAME 0.78 cores need 0.78 ROMs; 0.139 cores need 0.139 ROMs
python3 rom_audit.py --recheck --autofix --system mame
```

15.6 Track collection health over time

Save a snapshot before any major change (core update, new ROM batch) and compare afterwards:

```
cp rom_audit.csv snapshots/rom_audit_$(date +%Y%m%d).csv
# After changes:
python3 scripts/compare.py snapshots/rom_audit_20260601.csv rom_audit.csv
```

15.7 Find and remove duplicates

ROM sets often contain both the parent ROM and multiple regional clones. Some may work where others fail:

```
python3 scripts/duplicates.py --files --paths
# Shows duplicate names across systems so you can identify redundant copies
```

16 Investigating Genuine Errors

When a ROM is marked GENUINE ERROR it means autofix has tried every available core and none worked. This section guides you through diagnosing what is actually wrong.

16.1 Step 1: Read the error log

Each failing ROM has its emulator output archived:

```
# Batocera
cat
/userdata/system/rom_audit/audit_logs/mame/pacman.7z/es_launch_stderr.log

# RetroPie
cat ~/RetroPie/rom_audit/audit_logs/arcade/pacman.zip/es_launch_stderr.log
```

16.2 Step 2: Identify the error pattern

Log message pattern	Likely cause	Action
NOT FOUND: romname/filename.bin	Missing file inside ZIP/7z	Wrong ROM set version or incomplete dump
BIOS not found	Missing BIOS file	Install correct BIOS in bios directory
Wrong data-format or corrupt ROM	Corrupt download or bad dump	Re-download the ROM
NO GOOD DUMP KNOWN	Known bad dump in MAME DB	Find a different release of the same ROM
Failed to load content	ROM format incompatible with core	Try a different core version manually
colour emulation / video emulation	IMPERFECT driver (should be caught)	Check IMPERFECT_MARKERS list

16.3 Step 3: Check the ROM set version

MAME arcade cores are version-specific. The most common cause of GENUINE ERROR in arcade ROMs is a version mismatch between your ROM set and your core:

```
# Check which core versions are installed
ls /usr/lib/libretro/ | grep mame

# mame2003_libretro.so expects MAME 0.78 ROMs
# mame2010_libretro.so expects MAME 0.139 ROMs
# mame_libretro.so    expects current MAME ROMs
```

16.4 Step 4: Check for parent ROM

Clone ROMs depend on their parent ROM being present. If pacman.zip is marked GENUINE ERROR but puckman.zip works, you likely have the clone without the parent:

```
# Check if the parent is in your collection
ls /userdata/roms/mame/ | grep pacman
```

16.5 Step 5: Manual core test

Force a specific core to test a ROM manually without going through the audit:

```
python3 rom_audit.py --test "pacman.7z" --system mame --no-dashboard
```

16.6 When to give up

If a ROM is GENUINE ERROR after all investigation, accept it and quarantine it. Some ROMs have no good dump, require hardware that cannot be emulated, or are simply not supported by any available core. The audit tool's job is to identify these definitively, not to fix the unfixable.

```
python3 rom_audit.py --cleanup --action move --system mame --dry-run
python3 rom_audit.py --cleanup --action move --system mame
```

17 Long Audits and SSH Sessions

A full audit of 10,000+ ROMs takes many hours. Use tmux to protect against SSH disconnection.

```
# Start a named session
tmux new-session -s romaudit
python3 rom_audit.py --no-dashboard

# Detach without stopping: Ctrl+B then D
# Check progress from another terminal:
tmux attach -t romaudit

# Check the log file from outside tmux:
tail -f /userdata/system/rom_audit/rom_audit.log
```

ⓘ *A full Batocera collection of 8,000+ ROMs typically takes 10–16 hours depending on system speed and per-system timeouts. Recalbox with 6,000 ROMs ran in approximately 10 hours.*

18 Troubleshooting

18.1 Another audit is already running

```
ps aux | grep rom_audit
# If nothing is running, remove the stale PID file:
rm /userdata/system/rom_audit/rom_audit.pid     # Batocera
rm ~/RetroPie/rom_audit/rom_audit.pid            # RetroPie
rm /recalbox/share/system/rom_audit/rom_audit.pid # Recalbox
```

18.2 Screenshot shows loading screen not the game

```
python3 rom_audit.py --screenshot --screenshot-delay 6
```

18.3 All ROMs in a system failing

If an entire system fails with similar errors, check: is the BIOS installed? Is the correct ROM set version present? Run prereqs.py and check the error log of the first failing ROM.

18.4 MAME ROMs showing IMPERFECT instead of OK

This is expected behaviour. MAME's driver database flags these games as having known accuracy issues. The games are still playable. Use --include-imperfect with --cleanup only if you want to remove them.

18.5 Module import error after update

If you see ImportError after deploying new files, clear the Python bytecode cache:

```
find /userdata/system/rom_audit -name "*.pyc" -delete
find /userdata/system/rom_audit -name "__pycache__" -type d -exec rm -rf {} +
```

18.6 Atari 800 ROMs showing a blank blue screen

This is caused by the atari800_system core option being set to "400/800 (OS B)" instead of a compatible XL/XE or 800XL mode. Edit the file:

```
/opt/retropie/configs/all/retroarch-core-options.cfg
```

And set: atari800_system = "XL/XE (64K)" or atari800_system = "800XL (64K)"

18.7 RetroPie screenshots not working (no white flash)

RetroArch 1.19.1 on fresh installs defaults to a video driver that does not support the SCREENSHOT network command. The tool sets video_driver = "gl" in /dev/shm/retroarch.cfg automatically on each launch. If screenshots still fail, check that the global config has video_driver enabled:

```
grep "video_driver" /opt/retropie/configs/all/retroarch.cfg
```

If it shows # video_driver = "gl" (commented out), uncomment it. The tool patches this automatically on the next run.

18.8 Text adventure games (Zork, zmachine) timing out

Games launched with the CON: prefix (interactive terminal games like Zork via frotz) cannot be tested by this tool, as they require interactive stdin which is not available from SSH. The tool now detects these automatically and marks them MISSING CORE with a clear explanation, rather than waiting through the full timeout. No action needed.

18.9 GameMode warning in logs (RetroPie)

The message "GameMode cannot be enabled on this system" appears in RetroArch output on systems without the GameMode performance service installed. This is non-fatal, the tool ignores it and it does not affect results.

18.10 Games fail to launch with no display attached

On a Raspberry Pi, if the HDMI cable is unplugged or the display is shared with another device (for example, testing several boards in parallel against one shared monitor), the GPU may fail to establish a usable display mode at all. This can make every launch fail, not because of anything wrong with the ROMs, but because the emulator can never get a working display surface to render into.

Forcing HDMI hotplug makes the Pi behave as though a display is always connected, regardless of whether a cable is physically attached, by adding this line to config.txt (/boot/config.txt or /boot/firmware/config.txt depending on your OS layout):

```
hdmi_force_hotplug=1
```

ⓘ *This is a firmware-level setting read at boot, independent of Batocera/RetroPie/Recalbox, and only affects whether a display mode is established. It does not affect --screenshot, which captures the in-memory framebuffer rather than reading the physical cable, so screenshots keep working even while the cable is elsewhere. This is a configuration choice for your own setup, not something this tool currently detects or works around automatically.*

18.11 A wall of failures with no obvious cause (Raspberry Pi)

If an entire system (or an entire audit) comes back with an implausibly high error rate, far higher than a quick manual spot-check of a few of the failing ROMs would suggest. Sustained under-voltage during the run is worth ruling out before assuming the ROM set itself is the problem. This has been confirmed to cause exactly this pattern: widespread failures across ROMs that work fine once power is stable.

Check for it with:

```
vcgencmd get_throttled
```

This returns a hex bitmask. The bit that matters most after the fact is “under-voltage has occurred since boot”, a non-zero result there means it happened at some point during the audit, even if voltage has since recovered and the live status looks fine. A non-zero result generally means a higher-amperage power supply is the actual fix, not anything in the ROM collection or this tool.

ⓘ *vcgencmd is Raspberry Pi firmware tooling specifically. There isn’t a direct equivalent for x86 Batocera installs or other SBCs without their own vendor-specific power monitoring.*

18.12 A ROM I already fixed shows ERROR again after a fresh test (Batocera)

This looks alarming the first time you see it, but it is expected behaviour, not a regression. It happens specifically to ROMs that rely on your system-wide default core (for example mame.core=fbneo in batocera.conf) to work, rather than having their own per-game entry.

This tool deliberately disables your system-wide default core while testing, so that a core like FBNeo cannot quietly hide a bad dump behind what looks like a successful launch. Your per-game entries (written by a previous autofix run, or set up by hand) are never touched by this and are always respected. A ROM with no per-game entry of its own, that only worked because the system default handled it, will correctly fail a fresh test once the default is temporarily switched off for testing, even though it plays fine right now from the menu.

This is exactly why the tool is useful on a downloaded or pre-built image: the person who built it could not manually verify every ROM against the default core they chose. A tool that simply trusted that default wouldn't tell you anything you don't already believe. Run --recheck --autofix and it will find a working core and write a proper per-game entry, the same as it did the first time.

ⓘ *If you interrupt a recheck --autofix run partway through (closing the terminal, killing the process), a per-game entry can occasionally be left showing a core that was only being trialled rather than the one that actually ended up working. If a specific ROM looks like it regressed right after an interrupted run specifically, a clean --recheck --autofix re-run is the fix, as it tries every core fresh and writes whichever one actually works.*

18.13 Dreamcast or arcade-board (Flycast) ROMs failing with "Non-zero exit from launcher" (Batocera)

This message means the real launcher genuinely reported a failure exit code. It is not the tool killing a healthy game too early; that case is logged differently and would not produce this message at all. On dreamcast and its arcade-board siblings (naomi, naomi2, atomiswave, all sharing the same underlying Flycast engine), Flycast can need real time after being signalled to shut down before it exits cleanly; not waiting long enough produces exactly this exit code. The tool now waits longer specifically for these systems before reading final logs.

ⓘ This is fully resolved as of v1.4.4c. The fix has two parts: a deliberate post-kill delay giving the EGL/GLES graphics context time to release cleanly, and a new is_expected_audit_exit() check that distinguishes the tool's own deliberate kill of a running game from a genuine launcher failure. A full recheck of the affected system should now come back clean.

18.14 A result differs between two otherwise-identical runs, or only with --screenshot active

A failing emulator's own error text can occasionally take a brief moment to be written to its log files. Before accepting a clean OK result, the tool now waits a short moment and checks the logs a second time, specifically to catch error text that was still being written at the first check. This is automatic and needs no flag. If you notice a ROM taking marginally longer than before to report a result, this is very likely why, and it means the result is now more trustworthy, not slower for its own sake.

18.15 A genuine crash is reported inconsistently, ERROR on one run, OK on a re-run of the exact same ROM

If an emulator genuinely crashes right at the edge of its display window, whether that crash is caught depends on a single check happening a moment before or after that boundary passes, pure timing, not anything different about the ROM itself between runs. The tool now performs one additional check immediately after the display window ends, closing exactly that gap, so a genuine crash is now reported consistently every time rather than depending on this timing. This fixes the inconsistency in reporting; it does not explain why the underlying emulator crashed in the first place. That remains a separate question worth investigating on its own if a ROM keeps crashing.

19 Known Limitations

Limitation	Detail
Tape-based systems	ZX Spectrum, C64, Amstrad CPC report OK when the emulator launches. Tape loading happens inside the emulator and cannot be detected externally. Manual verification needed.
Sub-model configuration	Some games require specific hardware variant settings (e.g. Atari 800 atari800_system=130XE, VICE machine type, MSX variant). The tool reports OK because the emulator exits cleanly. Use --screenshot to catch wrong-hardware screens.
Standalone port visibility	Ports using standalone binaries (sdlpop, corsixth etc.) may not appear on screen during testing. The OK/ERROR result is still correct.
Gameplay testing	The tool confirms a game loads, not that it plays correctly. A game showing a garbled title screen would be recorded as OK.
IMPERFECT detection (Batocera only)	MAME warning strings are platform-specific. IMPERFECT detection is currently implemented for Batocera only. Recalbox and RetroPie do not expose these markers in the same way.
RetroArch standalone	Not supported. The tool works via platform launchers (emulatorlauncher on Batocera/Recalbox, runcommand on RetroPie). RetroArch standalone has a different architecture and is planned as a future platform.
Visually broken while running	A ROM can launch, stay running for the full display window, and be cleanly closed exactly as expected, while showing a frozen or garbled screen the whole time due to unemulated hardware (MAME's "005" Sega security board is a confirmed example). There is no log line for "the screen shows garbage", this is undetectable by design. Use --screenshot for manual visual review on ROMs you suspect of this.

20 Reporting a Bug

If you find a result that looks wrong, such as a ROM marked OK that does not actually work, or ERROR for something that should, the right evidence makes the difference between a quick fix and a lot of back and forth. This section covers what to gather before reporting an issue.

20.1 What to include

- The exact command you ran
- Platform and version, both are printed at the start of every run ("ROM Audit Tool vX.X.X starting") and shown on the dashboard while it runs
- What you expected versus what actually happened, specifically what you saw on the real screen if you watched it. Visual confirmation has repeatedly been the detail that distinguished a real bug from a result that was correct all along
- Whether you reproduced the same launch manually, outside the tool (the same emulator command run by hand, or via EmulationStation directly) and what that showed. This single piece of evidence has been the deciding factor in nearly every detection fix made to this tool, it is the most valuable thing you can include

20.2 Files that help most

Item	Location	Why it helps
CSV row	rom_audit/rom_audit.csv	The recorded status, elapsed time and notes for the exact ROM in question
Archived log directory	audit_logs/{system}/{rom}/	Contains the captured stdout/stderr for that exact test, and a screenshot file plus screenshot.txt if --screenshot was used
Live launch logs	See file locations table below, filenames differ per platform	The most recent raw output, in case the archived copy was cleared (passing ROMs only keep a screenshot, not full logs)
frontend.log (Recalbox only)	/recalbox/share/system/logs/frontend.log	EmulationStation's own native log, separate from anything the tool captures. Contains exact launch timing that has proven essential for diagnosing cases where a process exits quickly and cleanly but never actually loaded the game

20.3 Live log file locations

These are overwritten on every test, so grab them immediately after reproducing the issue:

```
Batocera:  /userdata/system/logs/es_launch_stdout.log, es_launch_stderr.log
RetroPie:  ~/RetroPie/rom_audit/retroarch_stdout.log, retroarch_stderr.log
Recalbox:  /recalbox/share/system/logs/es_launch_stdout.log,
es_launch_stderr.log
```

ⓘ *Recalbox's es_launch_stderr.log is always empty by design. Everything ends up in stdout. This is normal, not a sign something is missing.*

20.4 Pulling it together

A quick way to gather everything into one folder before attaching it to a GitHub issue:

```
mkdir bug_report
cp -r rom_audit/audit_logs/{system}/{rom} bug_report/
grep -i "romname" rom_audit/rom_audit.csv > bug_report/csv_row.txt
# Recalbox only, if available:
cp /recalbox/share/system/logs/frontend.log bug_report/
```

ⓘ *None of this is mandatory. A report with just the command and what you saw is still useful. But the more of the above you can include, the faster the actual cause gets found rather than guessed at.*

21 Appendix: File Locations

Item	Batocera	RetroPie	Recalbox
Tool root	/userdata/system/rom_audit/	~/RetroPie/rom_audit/	/recalbox/share/system/rom_audit/
Results CSV	rom_audit/rom_audit.csv	rom_audit/rom_audit.csv	rom_audit/rom_audit.csv
Error logs	rom_audit/audit_logs/	rom_audit/audit_logs/	rom_audit/audit_logs/
ROMs	/userdata/roms/	/home/pi/RetroPie/roms/	/recalbox/share/roms/
BIOS	/userdata/bios/	/home/pi/RetroPie/BIOS/	/recalbox/share/bios/
Quarantine	/userdata/faultyroms/	~/RetroPie/faultyroms/	/recalbox/share/faultyroms/

Author: Jason — muckypaws.com